

to lose the overview of them. You might be privy to dozens of hundreds of modules that can easily be categorized into different sub-headings. Think of it as a filing system – instead of having all of your files in a single directory, you can arrange them in different ones and get them organized according to the topics covered in each of the files. That's exactly how Packages in Python work. They allow you to organize modules into packages.

Packages

Modules are files that contain Python statements as well as definitions such as function and class definitions. A package, then, is basically a directory that contains Python files as well as a file with the name `_init_.py`. What this means is that every directory inside a Python path that consists of a file named `_init_.py` will be treated as a package when it comes to Python. It is completely possible for several modules to be put inside a Package.

Packages are a simple way of structuring Python's module namespace by applying “dotted module names”. A.B will stand for a submodule that is named B in a package that is named A. It is also possible for two different packages like P1 as well as P2 to have modules that have the same name - for instance A. The submodule A in the package P1 as well as the submodule A of the package P2 can be (and sometimes will be) totally different.

Here's a simple example of the same -

If you have a directory named `pkg` which contains two modules - `mod1.py` and `mod2.py`. The contents within these modules are as shown here: <https://dgit.in/PckgPy>. Keeping this structure in mind, if your `pkg` directory resides in a location where it can be found, you can refer to both the modules with the “dot” notation (`pkg.mod1`, `pkg.mod2`). This will allow you to import them while using the syntax that you are already familiar with, as shown here: <https://dgit.in/PckgPy2>. It is also possible to import modules using statements shown here: <https://dgit.in/PckgPy3>.

Technically speaking, you can import the package as well -

- `>>> import pkg`
- `>>> pkg`
- `<module 'pkg' (namespace)>`

However, this is not very useful. While technically speaking this is a syntactically correct Python statement, it does not have too much to contribute, particularly as far as placing any of the `pkg` modules into the local namespace is concerned, as shown in the code example that can be found at this link: <https://dgit.in/PckgPy4>.